

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

AIM : K-Nearest Neighbors (K-NN)

Implement the K-NN algorithm for classification or regression.

Apply the K-NN algorithm to a given dataset and predict the class or value for test data.

Evaluate the accuracy or error of the predictions and analyze the results.

CODE:

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
data = pd.read_csv("Dry_Bean_Dataset.csv")

# Display dataset information
print("=" * 60)
print("DATASET INFORMATION")
print("=" * 60)
print("Dataset Shape:", data.shape)
print("\nColumns:")
print(data.columns.tolist())
print("\nFirst 5 Records:")
print(data.head())

# -----
# Separate Features and Target
# -----

X = data.drop(columns=["Class"])
y = data["Class"]

# -----
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
# Split Dataset into Training and Testing Data
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

# -----
# Feature Scaling
# -----

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -----
# Apply K-NN Classifier
# -----

k = 5

knn = KNeighborsClassifier(n_neighbors=k)

knn.fit(X_train, y_train)

# -----
# Predict Test Data
# -----

y_pred = knn.predict(X_test)

# -----
# Evaluate Model
# -----
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
accuracy = accuracy_score(y_test, y_pred)
error = 1 - accuracy

print("\n" + "=" * 60)
print("K-NN CLASSIFICATION RESULTS")
print("=" * 60)

print("K Value:", k)
print("Accuracy:", round(accuracy * 100, 2), "%")
print("Error Rate:", round(error * 100, 2), "%")

# -----
# Classification Report
# -----

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# -----
# Confusion Matrix
# -----

print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))

# -----
# Display Sample Predictions
# -----

comparison = pd.DataFrame({
    "Actual Class": y_test.values[:10],
    "Predicted Class": y_pred[:10]
})

print("\n" + "=" * 60)
print("SAMPLE TEST DATA PREDICTIONS")
print("=" * 60)
print(comparison)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE
SUBJECT : AI

```
# -----  
# Accuracy for Different K Values  
# -----  
  
k_values = range(1, 16)  
accuracies = []  
  
for k_value in k_values:  
    model = KNeighborsClassifier(n_neighbors=k_value)  
    model.fit(X_train, y_train)  
    prediction = model.predict(X_test)  
    accuracies.append(accuracy_score(y_test, prediction))  
  
# -----  
# Accuracy vs K Graph  
# -----  
  
plt.figure(figsize=(8, 5))  
plt.plot(k_values, accuracies, marker="o")  
  
plt.title("K-NN Accuracy for Different K Values")  
plt.xlabel("Number of Neighbors (K)")  
plt.ylabel("Accuracy")  
plt.xticks(k_values)  
plt.grid(True)  
  
plt.show()  
  
# -----  
# Final Result Analysis  
# -----  
  
best_k = k_values[accuracies.index(max(accuracies))]  
best_accuracy = max(accuracies)  
  
print("\n" + "=" * 60)  
print("RESULT ANALYSIS")  
print("=" * 60)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

```
print("Initial K Value:", 5)
print("Initial Accuracy:", round(accuracy * 100, 2), "%")
print("Best K Value:", best_k)
print("Best Accuracy:", round(best_accuracy * 100, 2), "%")
print("Best Error Rate:", round((1 - best_accuracy) * 100, 2), "%")
```

OUTPUT:

The screenshot shows a Jupyter Notebook interface with the following content:

Code Cell:

```
print("Best K Value:", best_k)
print("Best Accuracy:", round(best_accuracy * 100, 2), "%")
print("Best Error Rate:", round((1 - best_accuracy) * 100, 2), "%")
```

Output:

0.7s

DATASET INFORMATION

Dataset Shape: (13611, 17)

Columns:

['Area', 'Perimeter', 'MajorAxisLength', 'MinorAxisLength', 'AspectRatio', 'Eccentricity', 'ConvexArea', 'EquivDiameter', 'Extent', 'Solidity', 'roundness', 'Compactness', 'ShapeFactor1', 'ShapeFactor2', 'ShapeFactor3', 'ShapeFactor4', 'Class']

First 5 Records:

	Area	Perimeter	MajorAxisLength	MinorAxisLength	AspectRatio	\
0	28395	610.291	208.178117	173.888747	1.197191	
1	28734	638.018	200.524796	182.734419	1.097356	
2	29380	624.110	212.826130	175.931143	1.209713	
3	30008	645.884	210.557999	182.516516	1.153638	
4	30140	620.134	201.847882	190.279279	1.060798	

	Eccentricity	ConvexArea	EquivDiameter	Extent	Solidity	roundness	\
0	0.549812	28715	190.141097	0.763923	0.988856	0.958027	
1	0.411785	29172	191.272751	0.783968	0.984986	0.887034	
2	0.562727	29690	193.410904	0.778113	0.989559	0.947849	
3	0.498616	30724	195.467062	0.782681	0.976696	0.903936	
4	0.333680	30417	195.896503	0.773098	0.990893	0.984877	

	Compactness	ShapeFactor1	ShapeFactor2	ShapeFactor3	ShapeFactor4	Class
0	0.913358	0.007332	0.003147	0.834222	0.998724	SEKER
1	0.953061	0.006979	0.003564	0.909051	0.998430	SEKER
2	0.908774	0.007244	0.003048	0.825871	0.999066	SEKER
3	0.928329	0.007017	0.003215	0.861794	0.994199	SEKER
4	0.970516	0.006697	0.003665	0.941900	0.999166	SEKER

K-NN CLASSIFICATION RESULTS

K Value: 5

Accuracy: 91.66 %

Error Rate: 8.34 %

Classification Report:

	precision	recall	f1-score	support
BARBUNYA	0.95	0.88	0.91	265
BOMBAY	1.00	1.00	1.00	104
CALI	0.92	0.94	0.93	326
DERMASON	0.91	0.91	0.91	709
HOROZ	0.95	0.95	0.95	386
SEKER	0.95	0.94	0.95	406
SIRA	0.84	0.87	0.86	527
accuracy			0.92	2723
macro avg	0.93	0.93	0.93	2723
weighted avg	0.92	0.92	0.92	2723

Confusion Matrix:

```
[[232  0 21  0  1  3  8]
 [  0 104  0  0  0  0  0]
 [  8  308  0  6  2  2]
 [  0  0 646  0 10 53]
 [  0  0  6  6 365  0  9]
 [  2  0  1  7  0 383 13]
```

K-NN Accuracy for Different K Values

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : AI

